VibePilot: Idea and Build Plan

HackIIITD finale, 24-25 October 2026. Mentor timings below are assumptions; confirm the real schedule with the organizers.

The idea in plain words

VibePilot is an assistant for people who build apps with AI tools like Lovable, Bolt, v0, Replit and Cursor. These tools produce a working app in minutes. Two problems then remain for the builder:

1. **They don't know what the AI built.** There are pages, features and third-party services they never chose and can't describe.
2. **The app doesn't work once it's live.** The AI says "now configure Google login in the console" or "add this environment variable on your host." These dashboards have no API, so someone has to click through them, and people get stuck. Typical symptoms are a login that works in preview and breaks on the real domain, or a payment webhook that never arrives.

VibePilot does the clicking for the user and proves the result. It understands the app, finds what's broken or unfinished, fixes it by operating the real dashboard in a browser, then re-tests and shows a receipt of exactly what changed.

Example: Aarav built a SaaS in Lovable, and clicking "Sign in with Google" shows `redirect_uri_mismatch`. VibePilot sees the error, reads the exact redirect address Google complained about, opens the Google Cloud console, proposes adding that address, waits for Aarav's approval, adds it, and re-tests the login. Aarav gets a receipt with screenshots.

What it isn't: it isn't a code scanner, a chatbot, or a static checklist. It acts on live websites with a human approving every sensitive change. That is where TinyFish matters, because the consoles it works in have no APIs.

Problem support (directional): one study found developers who delegated code generation to AI scored under 40% on comprehension tests, and a scan of 1,072 vibe-coded apps found a security flaw in 98%. Both figures come from outside vendors or blog sources.

The hero flow

This is the only thing that must work: App URL, Understand, Diagnose, Plan, Approve, Act, Verify, Receipt.

Step	What happens	TinyFish piece
1 Understand	Fetch the app's pages; the LLM writes an App Map: pages, features, services, data collected	Fetch
2 Diagnose	An agent clicks "Sign in with Google" and reports the error text and URL it sees	Agent (with output_schema)
3 Plan	The LLM turns the error into a fix: "add this redirect URI to the OAuth client"	none
4 Approve	The user sees the proposed change and clicks Approve	none
5 Act	An agent performs the edit in the Google Cloud console on a test project	Agent
6 Verify	A fresh run retries the login and checks that the error is gone	Agent / Browser
7 Receipt	A page shows what changed, with screenshots and timestamps	stored outputs

The agent calls run through /run-sse so the live trace streams to the screen.

Architecture (small on purpose)

- **Frontend:** Next.js with three screens: App Map, Live Run (trace plus approval button), Receipt.
- **Backend:** FastAPI (or Node) with SQLite and an SSE stream. The job moves through states: ANALYZING, DIAGNOSING, AWAITING_APPROVAL, EXECUTING, VERIFYING, DONE, or FAILED then GUIDED.
- **Endpoints:** POST /analyze, GET /jobs/{id}/stream, POST /jobs/{id}/approve, GET /jobs/{id}/receipt.
- **Evidence log:** every step stores tool, goal, result, timestamp and screenshot. The receipt is just a view of this log.

- **LLM:** structured JSON output for the App Map, the diagnosis, and the fix plan.

Present the bigger design (queue, provider "recipes", parallel runs) as the roadmap, not the build.

The demo target app

You need an app to point VibePilot at. Create it at the event as its own small repo called demo-target, deployed with Google login that is deliberately missing its redirect URI. It should take about 30 minutes. Whether you can use AI tools like Lovable to make it depends on the rules, so ask the mentors. If AI tools aren't allowed, write a minimal app by hand.

Build plan

Owners: **A** agent and TinyFish, **B** backend and LLM, **C** frontend, **D** product, target app and pitch. With fewer people, merge roles in order A+B and C+D.

Hours	A: TinyFish	B: Backend	C: Frontend	D: Product
0-1	API key, read Agent docs	Repo, FastAPI skeleton	Next.js skeleton	Lock scope, deploy demo-target
1-3	Fetch call; first Agent goal that clicks Sign in and reads the error	/analyze returns App Map JSON	URL box and App Map view	60-second pitch, flow diagram, run log
Mentor 1	Show URL in, App Map out, one TinyFish call			Ask about scoring and scope
3-8	Diagnose agent with output_schema; execute agent on the console	State machine, SSE, evidence log	Live Run screen with trace	Test accounts, console login plan
8-11	Verify step; first full run	Approval endpoint, fix-plan LLM prompt	Approval prompt, Receipt screen	Record first successful run

Hours	A: TinyFish	B: Backend	C: Frontend	D: Product
Mentor 2	Live run of the full flow			Take notes
11-16	Reliability: retries, better goals, guided-mode fallback	Error handling, cache results	Polish, error states	Run log numbers, README start
16-18	5 successful runs in a row	Cleanup	Final UI pass	Slides, architecture and TinyFish diagrams
Mentor 3	Pitch rehearsal			Collect the toughest questions
18-21	Bug fixes only			Backup video, final README
21-24	Feature freeze, final run, submit			Pitch rehearsal twice

Sleep in two shifts of about 3 hours between hours 13 and 20, so two people are always awake. Commit every 30-60 minutes.

Risks and fallbacks

Risk	Plan
Google console login hits 2FA or CAPTCHA from a cloud browser	Use a dedicated test Google account without 2FA, test it on venue Wi-Fi early, and use a saved browser context profile. If the live view lets a human take over, log in there. Verify this in the docs or with a TinyFish mentor.
The agent fails at the console edit	Guided mode: the agent opens the right page and highlights what to change; the user clicks. Switch to this at hour 12 if the edit still isn't reliable.

Risk	Plan
Google console is too hard	Swap to the Supabase "redirect URLs" setting or a Vercel env var, which usually have simpler logins.
Slow or failed runs on stage	Keep the backup video and a screenshot sequence.
Credits run out	Use Fetch and Search (free allowance) while building, and Agent runs only for the real flow.

Definition of done

1. The URL goes in and the App Map appears.
2. The agent finds the real error.
3. A proposed fix appears, and the approval gate works.
4. The console change is made, or guided mode completes it.
5. The re-test passes and the receipt shows screenshots.
6. At least 5 of the last 6 runs succeed, with the numbers logged.

Cut list

DNS, Stripe, multiple providers, repo analysis, user accounts, billing, and a polished dashboard. If one is left over after hour 18, add a second console recipe, such as the Supabase redirect URLs page, to show scalability.

Pitch line

"AI writes the app. VibePilot understands it, fixes the setup in the real dashboards, with your approval, and proves it works."